

BIRCH: Метод кластеризации

1. Основные концепции

BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies) - алгоритм иерархической кластеризации, который строит дерево признаков кластеризации (CF-дерево).

Ключевые компоненты:

Компонент	Описание
CF-вектор	Тройка (N, LS, SS) - количество точек, линейная сумма, квадратичная сумма
Порог (threshold)	Максимальный радиус подкластера
Коэффициент ветвления	Максимальное количество подкластеров в узле

2. Реализация на Python

3. Ключевые выводы и рекомендации

Когда использовать BIRCH:

- **Большие датасеты** (эффективен по памяти)
- **Некруглые кластеры** (лучше чем K-Means)
- **Инкрементальное обучение** (partial_fit)
- **Предобработка данных** (сжатие в подкластеры)

Ограничения:

- Не для данных высокой размерности (>20 признаков)
- Чувствителен к масштабированию признаков
- Порог (threshold) критически важен

Рекомендации по параметрам:

- **threshold:** начинайте с 0.5, увеличивайте для больших кластеров
- **branching_factor:** 50-100 для большинства задач
- **n_clusters:** None для анализа подкластеров, число кластеров для финальной кластеризации

Метрики оценки:

- **Silhouette Score:** компактность и делимость кластеров

- **Adjusted Rand Index:** соответствие истинным меткам
- **Визуальный анализ:** всегда проверяйте результаты

1. Базовая реализация

Установка библиотек

pip install scikit-learn numpy matplotlib pandas

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
from sklearn.cluster import Birch
```

```
from sklearn.datasets import make_blobs, make_circles
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import silhouette_score, adjusted_rand_score
```

2. Простые примеры. Пример 1: Синтетические данные (blobs)

Генерация данных

```
X, y_true = make_blobs(n_samples=500, centers=4,  
                        cluster_std=1.0, random_state=42)
```

Масштабирование

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

BIRCH кластеризация

```
birch = Birch(threshold=0.5, branching_factor=50, n_clusters=4)
```

```
y_pred = birch.fit_predict(X_scaled)
```

Визуализация

```
plt.figure(figsize=(12, 4))
```

```
plt.subplot(1, 2, 1)
```

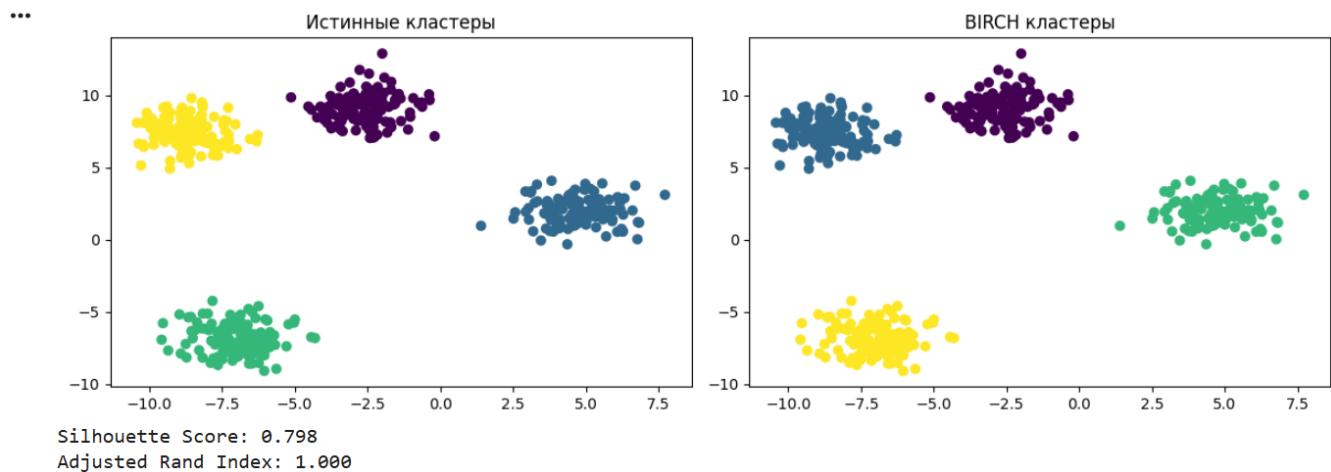
```
plt.scatter(X[:, 0], X[:, 1], c=y_true, cmap='viridis')
```

```
plt.title('Истинные кластеры')
```

```
plt.subplot(1, 2, 2)
```

```
plt.scatter(X[:, 0], X[:, 1], c=y_pred, cmap='viridis')
plt.title('BIRCH кластеры')
plt.tight_layout()
plt.show()

# Оценка
print(f'Silhouette Score: {silhouette_score(X_scaled, y_pred):.3f}')
print(f'Adjusted Rand Index: {adjusted_rand_score(y_true, y_pred):.3f}')
```



Пример 2: Нелинейные кластеры

```
# Генерация концентрических окружностей
X, y_true = make_circles(n_samples=500, factor=0.5, noise=0.05,
random_state=42)

# BIRCH кластеризация
birch1 = Birch(threshold=0.3, n_clusters=2)
y_pred1 = birch1.fit_predict(X)

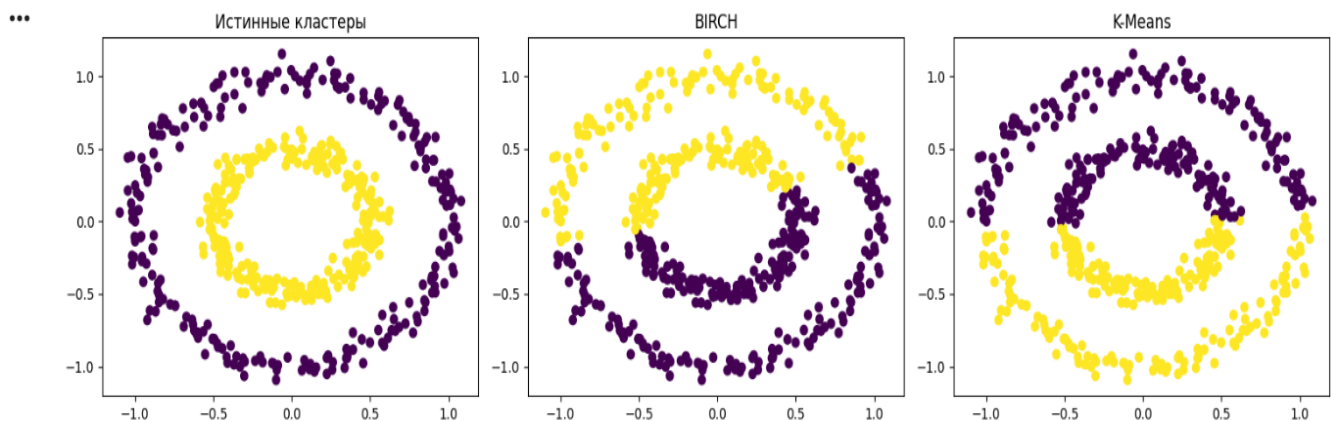
# K-Means для сравнения
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=2, random_state=42)
y_pred2 = kmeans.fit_predict(X)

# Визуализация
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
axes[0].scatter(X[:, 0], X[:, 1], c=y_true, cmap='viridis')
```

```

axes[0].set_title('Истинные кластеры')
axes[1].scatter(X[:, 0], X[:, 1], c=y_pred1, cmap='viridis')
axes[1].set_title('BIRCH')
axes[2].scatter(X[:, 0], X[:, 1], c=y_pred2, cmap='viridis')
axes[2].set_title('K-Means')
plt.tight_layout()
plt.show()

```



3. Задачи для проверки модели. Задача 1: Подбор порога (threshold)

```

def optimize_threshold(X, threshold_range):
    """Подбор оптимального порога"""
    scores = []
    models = []
    for thresh in threshold_range:
        birch = Birch(threshold=thresh, n_clusters=None)
        y_pred = birch.fit_predict(X)
        if len(np.unique(y_pred)) > 1: # Проверка наличия нескольких кластеров
            score = silhouette_score(X, y_pred)
            scores.append(score)
            models.append(birch)
            print(f"Threshold={thresh:.2f}: {len(np.unique(y_pred))} кластеров,
                  Silhouette={score:.3f}")
    else:

```

```

scores.append(-1)
models.append(None)
print(f"Threshold={thresh:.2f}: 1 кластер")

# Лучшая модель
best_idx = np.argmax(scores)
return threshold_range[best_idx], models[best_idx], scores

# Пример использования
thresholds = [0.1, 0.3, 0.5, 0.7, 1.0]
best_thresh, best_model, scores = optimize_threshold(X_scaled, thresholds)
print(f"\nЛучший threshold: {best_thresh}")

Threshold=0.10: 57 кластеров, Silhouette=0.342
Threshold=0.30: 6 кластеров, Silhouette=0.553
Threshold=0.50: 5 кластеров, Silhouette=0.681
Threshold=0.70: 3 кластеров, Silhouette=0.747
Threshold=1.00: 3 кластеров, Silhouette=0.747

```

Лучший threshold: 0.7

Задача 2: Влияние масштабирования

```

# Данные с разными масштабами
X_unscaled, y = make_blobs(n_samples=300, centers=3,
                           cluster_std=[1.0, 2.0, 3.0], random_state=42)

# Без масштабирования
birch_unscaled = Birch(threshold=1.0, n_clusters=3)
y_pred_unscaled = birch_unscaled.fit_predict(X_unscaled)

# С масштабированием
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_unscaled)
birch_scaled = Birch(threshold=1.0, n_clusters=3)
y_pred_scaled = birch_scaled.fit_predict(X_scaled)

```

Сравнение

```
print("Без масштабирования:")
```

```
print(f'Silhouette: {silhouette_score(X_unscaled, y_pred_unscaled):.3f}')
```

```
print(f'ARI: {adjusted_rand_score(y, y_pred_unscaled):.3f}')
```

```
print("\nС масштабированием:")
```

```
print(f'Silhouette: {silhouette_score(X_scaled, y_pred_scaled):.3f}')
```

```
print(f'ARI: {adjusted_rand_score(y, y_pred_scaled):.3f}')
```

Без масштабирования: Silhouette: 0.384 ARI: 0.478

С масштабированием: Silhouette: 0.599 ARI: 0.545

4. Работа с реальными данными

```
from sklearn.datasets import load_iris, load_wine
```

```
# Загрузка данных Iris
```

```
iris = load_iris()
```

```
X_iris = iris.data
```

```
y_iris = iris.target
```

```
# Масштабирование
```

```
scaler = StandardScaler()
```

```
X_iris_scaled = scaler.fit_transform(X_iris)
```

```
# BIRCH кластеризация
```

```
birch_iris = Birch(threshold=0.8, branching_factor=50, n_clusters=3)
```

```
y_iris_pred = birch_iris.fit_predict(X_iris_scaled)
```

```
# Оценка
```

```
print("=== Данные Iris ===")
```

```
print(f'Silhouette Score: {silhouette_score(X_iris_scaled, y_iris_pred):.3f}')
```

```
print(f'Adjusted Rand Index: {adjusted_rand_score(y_iris, y_iris_pred):.3f}')
```

```
# Сравнение с K-Means
```

```
from sklearn.cluster import KMeans
```

```
kmeans = KMeans(n_clusters=3, random_state=42)
```

```
y_kmeans = kmeans.fit_predict(X_iris_scaled)
```

```
print(f"\nK-Means ARI: {adjusted_rand_score(y_iris, y_kmeans):.3f}")
```

==== Данные Iris ====

Silhouette Score: 0.455 Adjusted Rand Index: 0.640 K-Means ARI: 0.433

5. Интерактивная проверка модели

```
def birch_analysis(X, y_true=None):  
    """  
    Полный анализ BIRCH кластеризации  
    """  
  
    # Масштабирование  
    scaler = StandardScaler()  
    X_scaled = scaler.fit_transform(X)  
  
    # Тестирование разных параметров  
    thresholds = [0.2, 0.5, 0.8, 1.2]  
    branch_factors = [20, 50, 100]  
    results = []  
  
    for thresh in thresholds:  
        for bf in branch_factors:  
            birch = Birch(threshold=thresh, branching_factor=bf, n_clusters=None)  
            y_pred = birch.fit_predict(X_scaled)  
            n_clusters = len(np.unique(y_pred))  
            if n_clusters > 1:  
                silhouette = silhouette_score(X_scaled, y_pred)  
                if y_true is not None:  
                    ari = adjusted_rand_score(y_true, y_pred)  
                else:  
                    ari = None  
            results.append({  
                'threshold': thresh,  
                'branching_factor': bf,
```

```

        'n_clusters': n_clusters,
        'silhouette': silhouette,
        'ari': ari
    })
    print(f"th={thresh}, bf={bf}: {n_clusters} кластеров, "
          f"sil={silhouette:.3f}" +
          (f", ari={ari:.3f}" if ari else ""))

# Лучший результат по силуэту
if results:
    best = max(results, key=lambda x: x['silhouette'])
    print("\n=== Лучшая модель ===")
    print(f"Threshold: {best['threshold']}")
    print(f"Branching factor: {best['branching_factor']}")
    print(f"Кластеров: {best['n_clusters']}")
    print(f"Silhouette: {best['silhouette']:.3f}")
    if best['ari']:
        print(f"ARI: {best['ari']:.3f}")

    return results

```

Пример использования

```
X, y = make_blobs(n_samples=300, centers=4, random_state=42)
```

```
results = birch_analysis(X, y)
```

```
th=0.2, bf=20: 13 кластеров, sil=0.344, ari=0.467
```

```
th=0.2, bf=50: 13 кластеров, sil=0.344, ari=0.467
```

```
th=0.2, bf=100: 13 кластеров, sil=0.344, ari=0.467
```

```
th=0.5, bf=20: 4 кластеров, sil=0.797, ari=0.991
```

```
th=0.5, bf=50: 4 кластеров, sil=0.797, ari=0.991
```

```
th=0.5, bf=100: 4 кластеров, sil=0.797, ari=0.991
```

```
th=0.8, bf=20: 3 кластеров, sil=0.744, ari=0.712
```

```
th=0.8, bf=50: 3 кластеров, sil=0.744, ari=0.712
```


th=0.8, bf=100: 3 кластеров, sil=0.744, ari=0.712

th=1.2, bf=20: 3 кластеров, sil=0.744, ari=0.712

th=1.2, bf=50: 3 кластеров, sil=0.744, ari=0.712

th=1.2, bf=100: 3 кластеров, sil=0.744, ari=0.712

==== Лучшая модель ====

Threshold: 0.5

Branching factor: 20

Кластеров: 4

Silhouette: 0.797

ARI: 0.991

6. Визуализация CF-дерева

```
def plot_birch_structure(X, threshold=0.5):
```

```
    """
```

```
    Визуализация подкластеров BIRCH
```

```
    """
```

```
    from sklearn.cluster import Birch
```

```
    # Обучение с сохранением подкластеров
```

```
    birch = Birch(threshold=threshold, branching_factor=50, n_clusters=None)
```

```
    birch.fit(X)
```

```
    # Получение подкластеров
```

```
    subclusters = birch.subcluster_centers_
```

```
    # Визуализация
```

```
    plt.figure(figsize=(10, 6))
```

```
    # Исходные точки
```

```
    plt.scatter(X[:, 0], X[:, 1], c='lightgray', alpha=0.5, s=20, label='Точки')
```

```
    # Подкластеры
```

```
    if subclusters is not None:
```

```
        plt.scatter(subclusters[:, 0], subclusters[:, 1],
```

```

c='red', s=100, marker='*', label='Центры подкластеров')

plt.title(f'BIRCH подкластеры (threshold={threshold})')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print(f'Количество подкластеров: {len(subclusters)}')

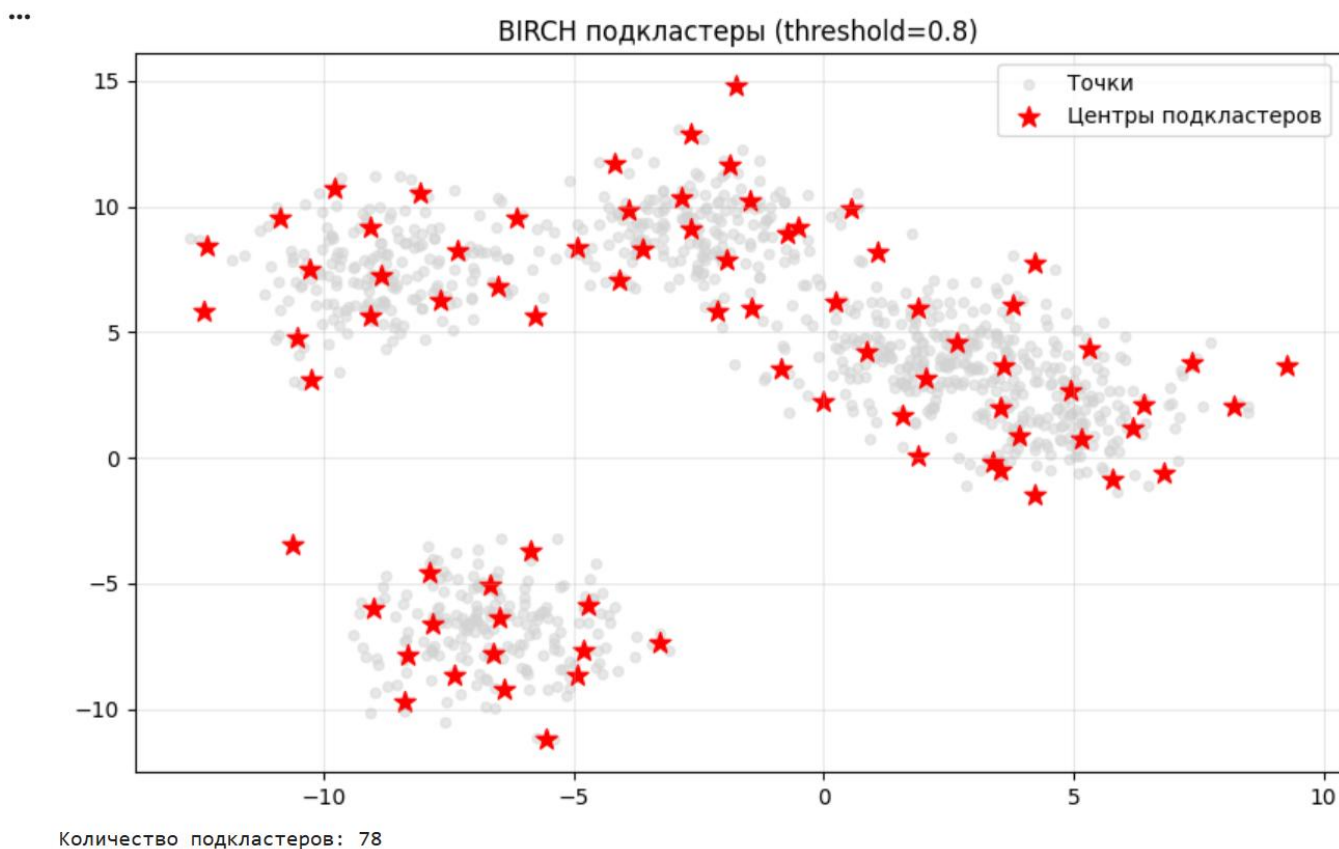
return subclusters

# Пример

X, _ = make_blobs(n_samples=1000, centers=5, cluster_std=1.5,
random_state=42)

subclusters = plot_birch_structure(X, threshold=0.8)

```



Ключевые выводы BIRCH эффективен для больших данных и нелинейных кластеров. Масштабирование обязательно - иначе метрики расстояния искажаются. Порог (**threshold**) - главный параметр, влияющий на число кластеров. **Silhouette Score** - лучшая метрика при отсутствии истинных меток. Сравнивайте с **K-Means** для понимания преимуществ BIRCH